

Atividade Continuada — Abstração

Orientação a Objetos em Java

Aplique os conceitos de abstração criando classes que modelem objetos do mundo real. Para cada exercício, instancie os objetos no método main e teste todos os comportamentos descritos.

Exercício 1 — Sistema de Controle de Medicamentos

Você foi contratado para desenvolver um sistema de controle de estoque para uma farmácia. Para isso, deverá criar uma classe chamada `Medicamento`, que represente um produto medicinal disponível para venda.

ABSTRAÇÃO

A classe deve conter os seguintes atributos:

- `nome` (nome comercial do medicamento)
- `principioAtivo` (substância ativa do medicamento)
- `quantidadeEstoque` (unidades disponíveis em estoque)
- `precoUnitario` (preço de venda por unidade)
- `necessitaReceita` (indica se o medicamento exige receita médica)

Todos os atributos devem ser definidos de forma apropriada para que representem corretamente as informações de um medicamento.

ENCAPSULAMENTO

Os atributos devem ser privados, sendo seu acesso e alteração realizados apenas pelos métodos obrigatórios.

MÉTODOS OBRIGATÓRIOS

- **`entradaEstoque(quantidade)`**: adiciona unidades ao estoque. Não deve permitir valores negativos ou zero.
- **`venderUnidades(quantidade)`**: subtrai a quantidade do estoque. Não deve permitir a venda se o estoque for insuficiente.
- **`aplicarReajuste(percentual)`**: reajusta o preço unitário pelo percentual informado. Não deve permitir percentuais negativos ou acima de 100%.
- **`exibirInformacoes()`**: imprime o nome, princípio ativo, quantidade em estoque, preço unitário e se necessita de receita.

Exercício 2 — Sistema de Gerenciamento de Alunos de Academia

Você foi contratado para desenvolver um sistema de gestão de alunos para uma academia de musculação. Para isso, deverá criar uma classe chamada `AlunoAcademia`, que represente um aluno matriculado.

ABSTRAÇÃO

A classe deve conter os seguintes atributos:

- `nome` (nome completo do aluno)
- `matricula` (código único de identificação do aluno)
- `planoMensal` (nome do plano contratado: BÁSICO, INTERMEDIÁRIO, PREMIUM)
- `totalAulasRealizadas` (contador de aulas que o aluno já realizou no mês)
- `mensalidadeAtiva` (indica se a mensalidade está em dia)

Todos os atributos devem ser definidos de forma apropriada para que representem corretamente as informações de um aluno.

ENCAPSULAMENTO

Os atributos devem ser privados, sendo seu acesso e alteração realizados apenas pelos métodos obrigatórios.

MÉTODOS OBRIGATÓRIOS

- **`registrarAula()`**: incrementa o total de aulas realizadas em 1. Só deve registrar a aula se a mensalidade estiver ativa. Caso contrário, deve exibir uma mensagem informando que o acesso está bloqueado por inadimplência.
- **`renovarMensalidade()`**: marca a mensalidade como ativa e reinicia o contador de aulas realizadas para zero.
- **`suspenderMensalidade()`**: marca a mensalidade como inativa. Não deve ser possível suspender uma mensalidade que já esteja inativa.
- **`exibirInformacoes()`**: imprime o nome, matrícula, plano contratado, total de aulas realizadas no mês e status da mensalidade.

Exercício 3 — Sistema de Reserva de Passagens Aéreas

Você foi contratado para desenvolver um sistema de reserva de assentos para uma companhia aérea. Para isso, deverá criar uma classe chamada `Passagem`, que represente uma passagem aérea emitida para um passageiro.

ABSTRAÇÃO

A classe deve conter os seguintes atributos:

- `codigoReserva` (identificador único da reserva)
- `nomePassageiro` (nome completo do passageiro)
- `origem` (sigla do aeroporto de origem)
- `destino` (sigla do aeroporto de destino)
- `valorTarifa` (preço base da passagem em reais)
- `dividaExistente` (valor pendente de pagamento referente taxas geradas)
- `statusReserva` (situação atual: `CONFIRMADA`, `CANCELADA` ou `AGUARDANDO`)

Todos os atributos devem ser definidos de forma apropriada para que representem corretamente as informações de uma passagem.

ENCAPSULAMENTO

Os atributos devem ser privados, sendo seu acesso e alteração realizados apenas pelos métodos obrigatórios.

MÉTODOS OBRIGATÓRIOS

- **`confirmarReserva()`**: altera o status da reserva para `CONFIRMADA`. Não deve ser possível confirmar uma reserva que já esteja `CANCELADA`.
- **`cancelarReserva()`**: altera o status para `CANCELADA`. Se a reserva já estiver `CONFIRMADA`, aplica-se uma multa de 15%, armazenando o valor como dívida a ser cobrada do passageiro. Não deve ser possível cancelar uma reserva que já esteja `CANCELADA`.
- **`aplicarUpgrade(valorAdicional)`**: adiciona o valor informado à tarifa, representando uma melhoria de classe. Só deve ser permitido se a reserva estiver com status `CONFIRMADA` ou `AGUARDANDO`. Adiciona o valor adicional também como dívida a ser cobrada do passageiro. Não aceita valores negativos.
- **`exibirInformacoes()`**: imprime o código da reserva, nome do passageiro, trecho (origem → destino), valor da tarifa e status da reserva.